LAB

Implement a document-based RAG system to enhance LLM responses

IBM SkillsBuild

Contents

Introduction

In this lab, you'll implement a document-based RAG system using Google Colab to enhance large language model (LLM) responses.

Software requirements

To complete this lab, you don't have to install any software on your computer. You only require a Google account to access Google Colab and a Replicate account to access and use various artificial intelligence (AI) models.

Objective

After completing this lab, you should be able to:

- Implement a document-based RAG system to enhance LLM responses

Lab steps

This lab requires you to complete the following procedures:

- Create a GitHub account
- Create a Replicate account
- Sign up for Google Colab
- Prepare the Google Colab notebook with the necessary libraries
- Configure and test the IBM Granite model for baseline responses
- Set up a RAG system for context-based answering
- Compare baseline responses with context-augmented answers

Estimated duration to complete

30 minutes

Scenario
Background information

GoTravel is an online travel agency that helps customers plan and book flights, hotels, and holiday packages. The agency’s goal is to make travel planning simple, fast, and reliable.

Challenge

However, GoTravel’s customer support team struggles to provide timely and accurate answers to customer queries. Customers often wait too long for answers to policy-related questions, and even then, the information is sometimes incomplete or inconsistent. This leads to repeated inquiries, higher support costs, and growing customer dissatisfaction. If unresolved, these issues threaten GoTravel’s reputation and customer loyalty.

Solution

To address these issues, GoTravel plans to deploy a RAG-enabled travel assistant. The assistant will provide timely, accurate, and context-aware answers to customer queries, reducing delays, cutting costs, and improving customer satisfaction.

You are leading the team responsible for building the Travel Assistant. Your challenge is to design an assistant that uses the company’s internal travel policy documents to provide reliable, trustworthy answers. To achieve this, you’ll implement a document-based RAG system that retrieves relevant policy information and enriches the LLM’s responses with accurate context.

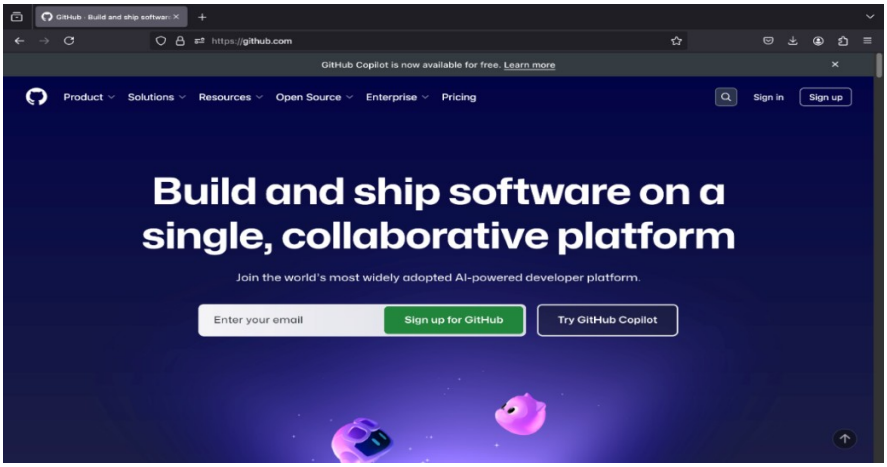
Create a GitHub account

Overview

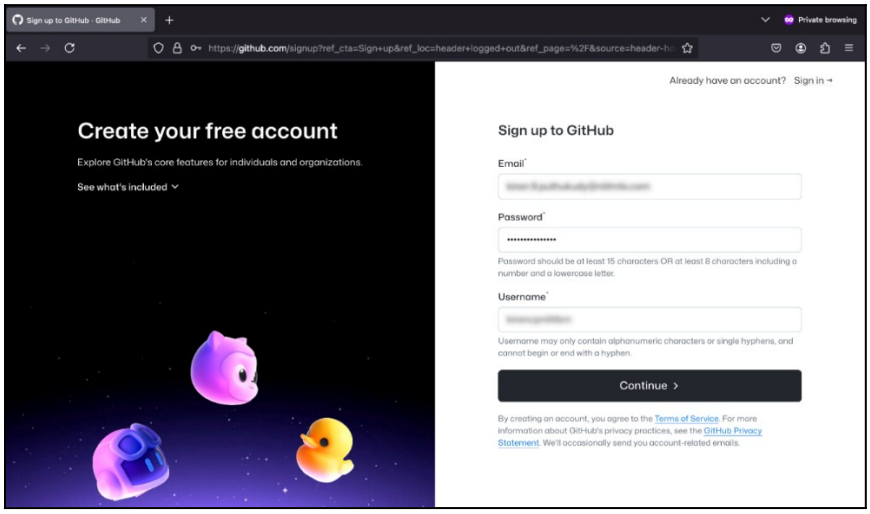
In this procedure, you'll set up a GitHub account. GitHub is a platform that helps developers store, manage, and share code, while also supporting collaboration through tools such as version control, bug tracking, and task management. Setting up a GitHub account provides access to the Replicate platform, which is required to complete the lab efficiently.

Instructions

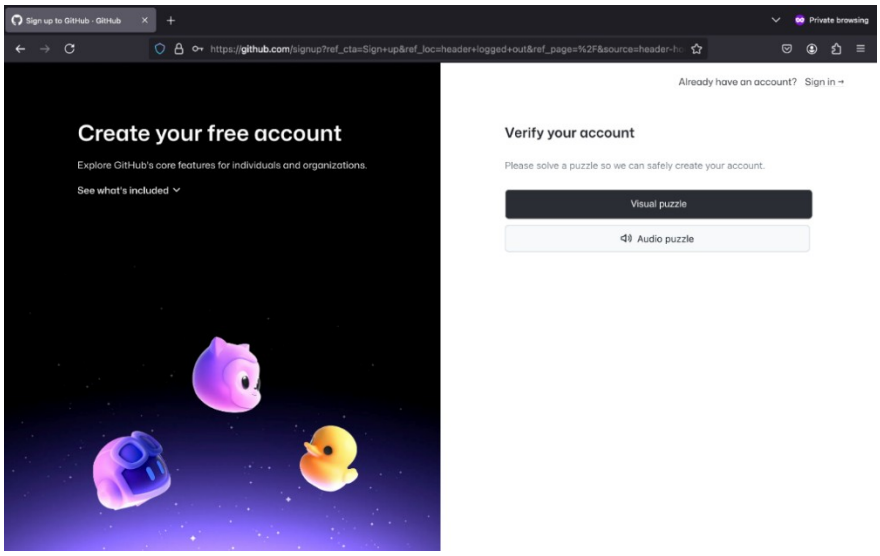
1. To create a GitHub account, go to the GitHub website.
2. Select the **Sign up for GitHub** button, as shown in the following screenshot.



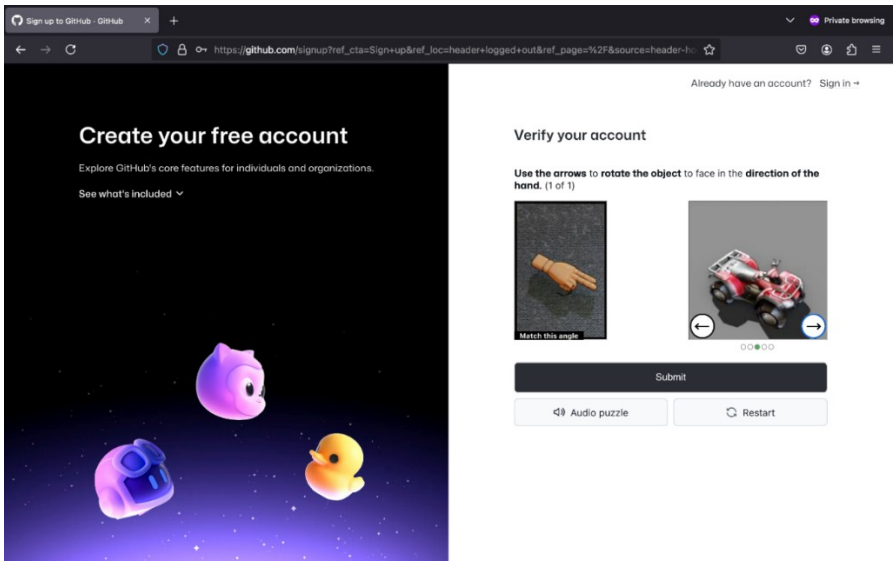
3. Enter your details in the **Email**, **Password**, and **Username** fields. Then, select **Continue**. The following screenshot shows the "Sign up to GitHub" page.



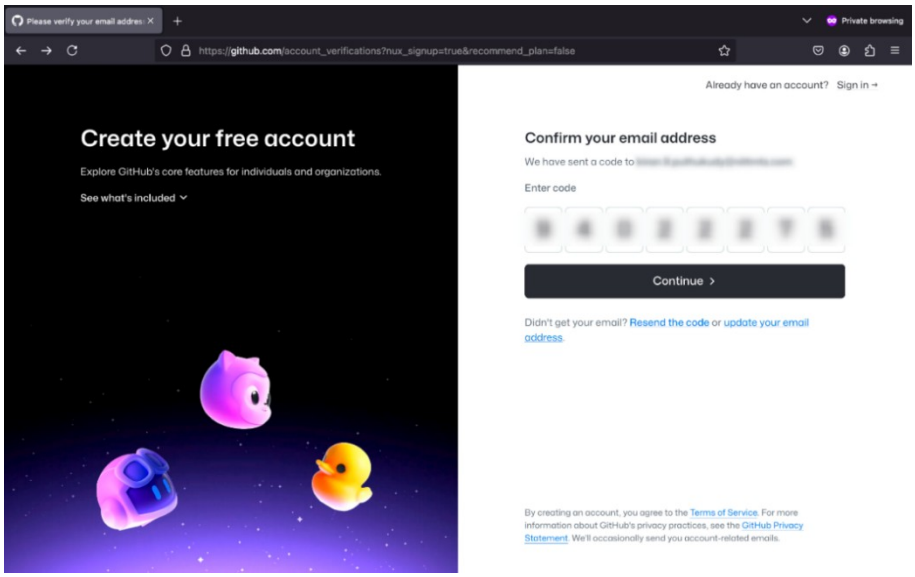
4. To verify your account, select the **Visual puzzle** button.



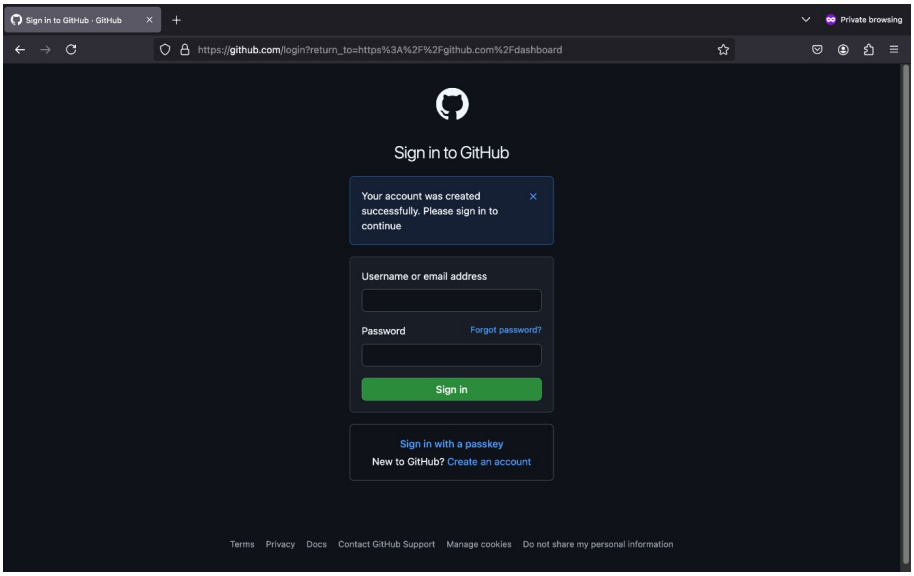
5. Solve the visual puzzle and select **Submit**.



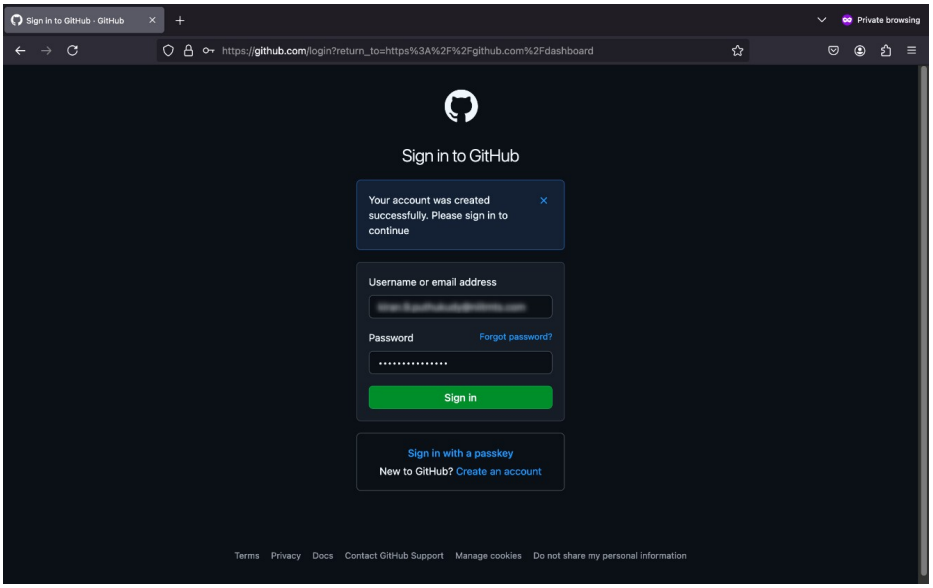
6. To confirm your email, enter the confirmation code sent to your registered email in the **Enter code** field and select **Continue**.



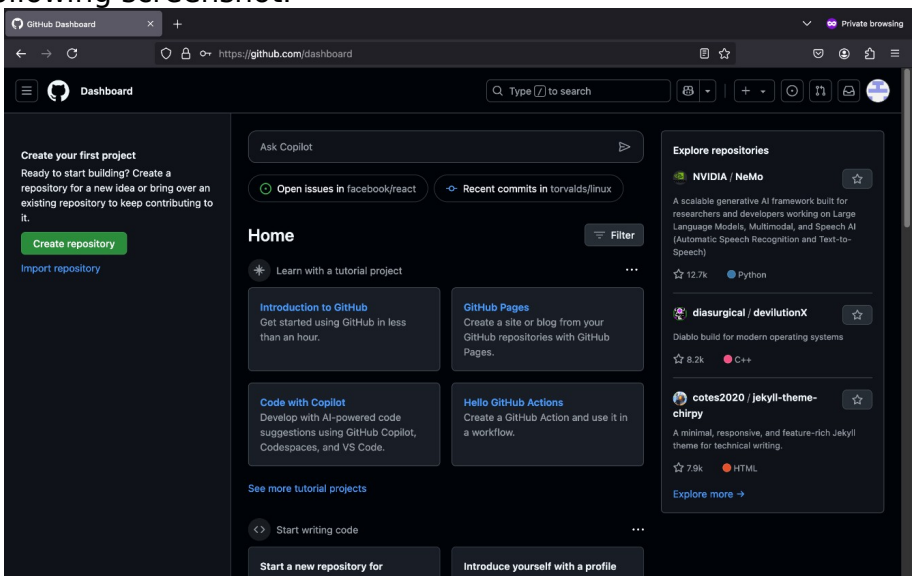
After your GitHub account has been successfully created, a confirmation message is displayed as shown in the following screenshot.



7. To sign in to your account, enter your credentials in the **Username or email address** and **Password** fields, and then select **Sign in**. The following screenshot shows the GitHub sign-in page which includes fields for entering the username and password.



8. After you sign in, the GitHub dashboard appears as shown in the following screenshot.



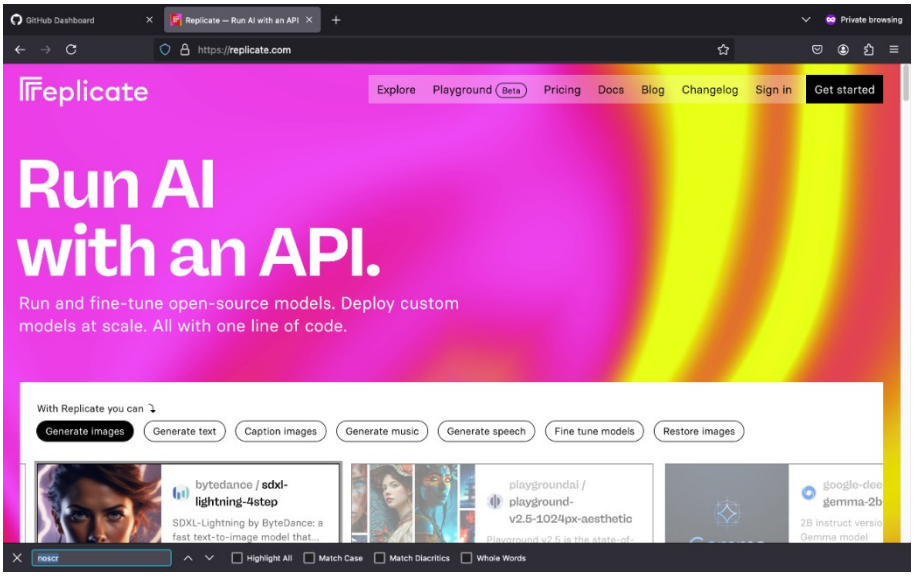
Create a Replicate account

Overview

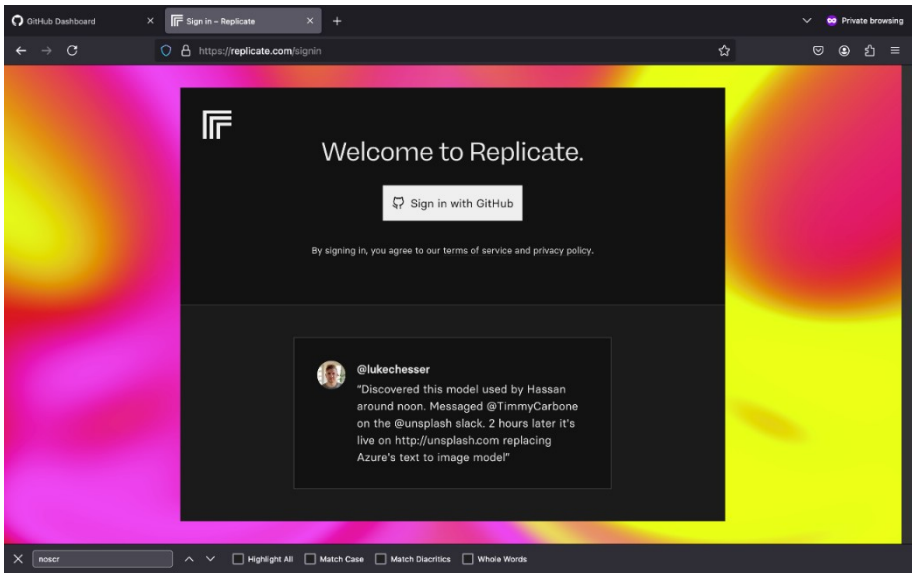
In this procedure, you'll use your GitHub account to register for a Replicate account. Replicate is a cloud-based platform that lets you use AI models such as IBM Granite without requiring advanced hardware. As part of this procedure, you'll create a Replicate token. A token is a secure access key that allows the lab environment to authenticate with Replicate and run models from your account in Google Colab.

Instructions

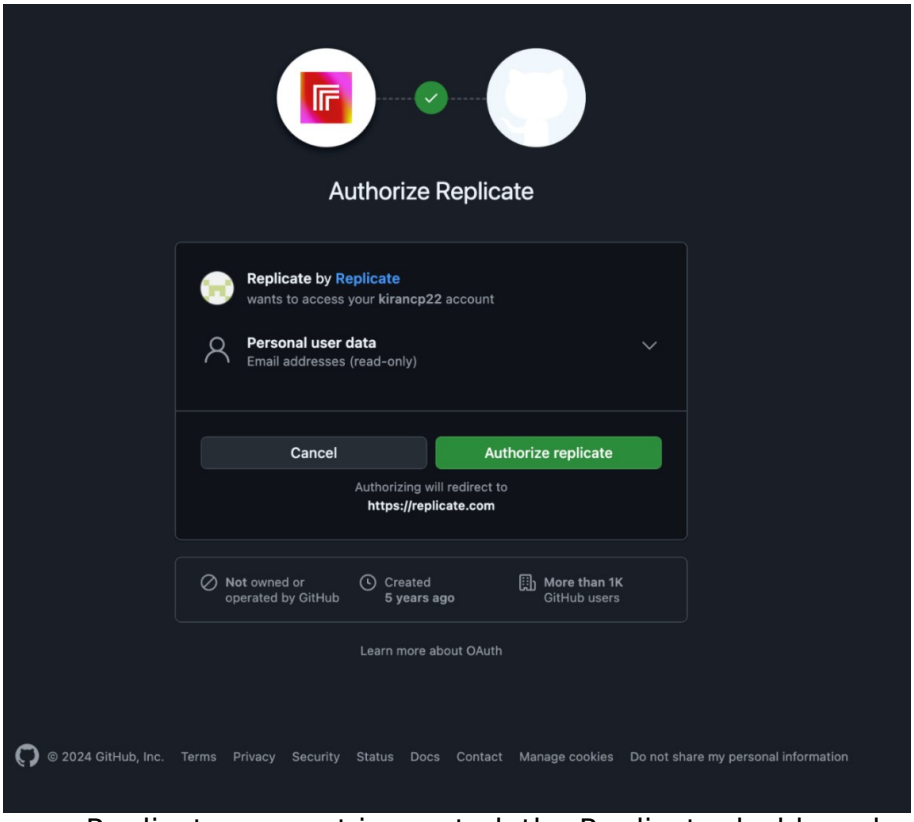
1. Go to the Replicate website.
2. Select **Get started**, as shown in the following screenshot.



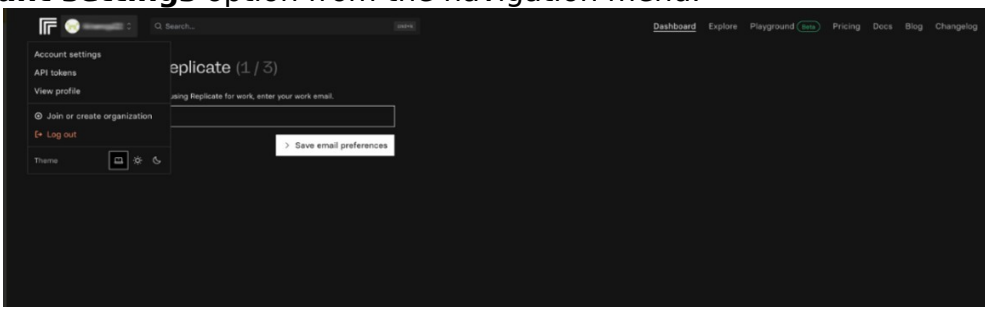
3. Select the **Sign in with GitHub** button, as shown in the following screenshot.



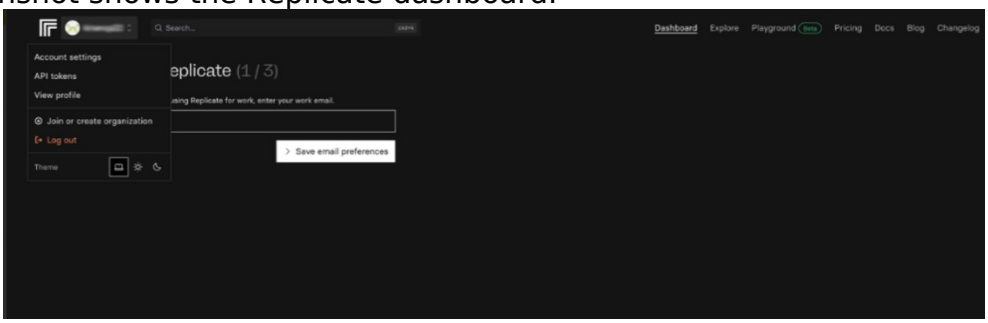
4. Select **Authorize replicate** to continue, as shown in the following screenshot.



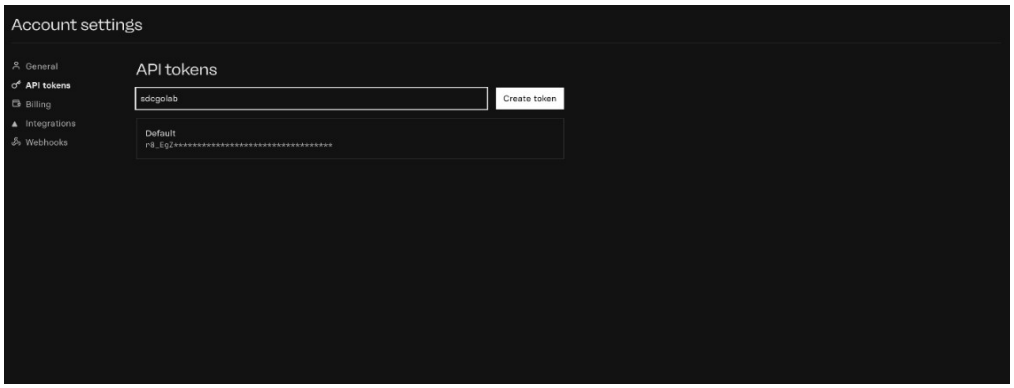
5. After your Replicate account is created, the Replicate dashboard appears as shown in the following screenshot. To create a Replicate token, select the **Account settings** option from the navigation menu.



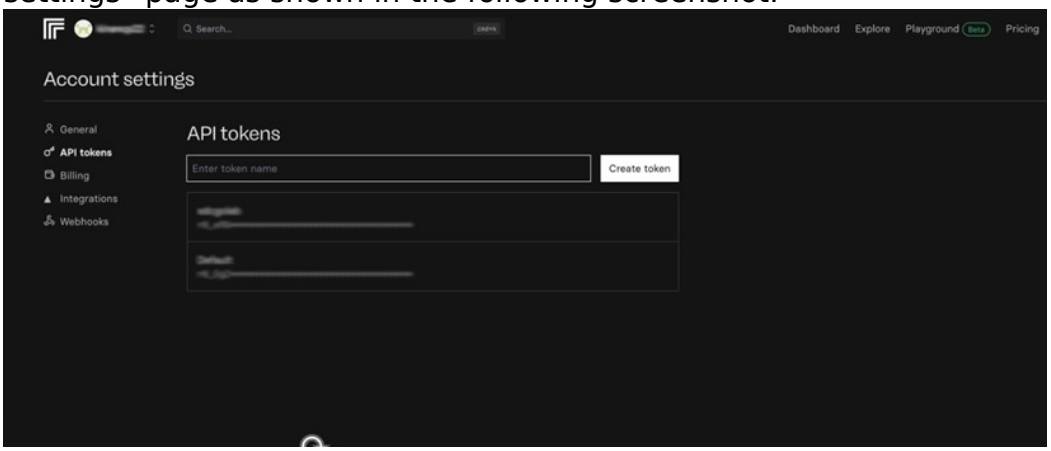
6. Select **API tokens** from the **Account settings** panel. The following screenshot shows the Replicate dashboard.



7. Enter a name for the token in the **API tokens** field and then select **Create token**, as shown in the following screenshot.

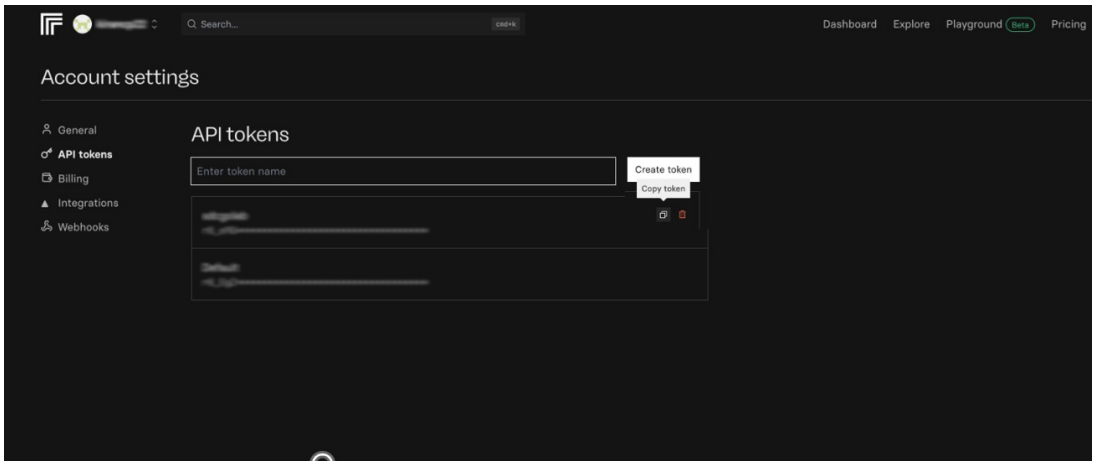


After your API token is created, it should be displayed on the “Account settings” page as shown in the following screenshot.

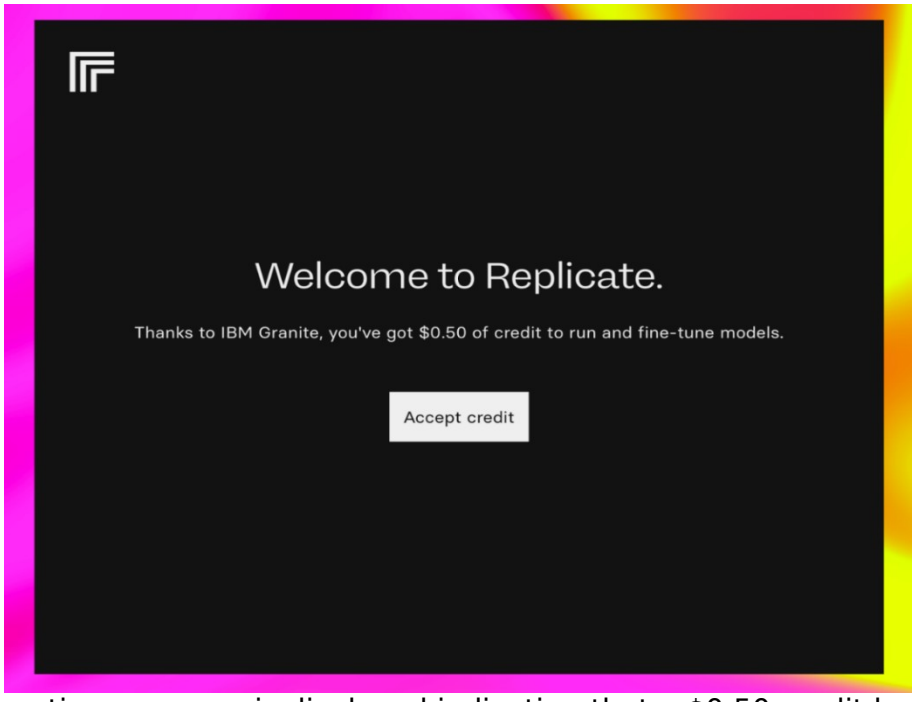


8. To copy the Replicate API, select the **Copy token** icon, as shown in the following screenshot.

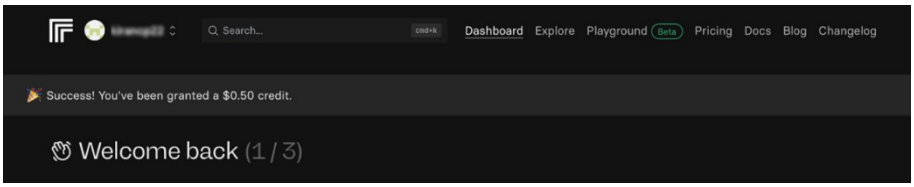
   Note: Save the Replicate token because you’ll need it to authenticate with the Replicate API when running code in the Google Colab environment later in this lab.



9. To run the lab without interruptions, you’ll require some Replicate credits. Go to the Replicate invite link to claim your free $0.50 credit.
10. To claim the amount, select **Accept credit**.



A confirmation message is displayed indicating that a $0.50 credit has been added to your Replicate account as shown in the following screenshot.



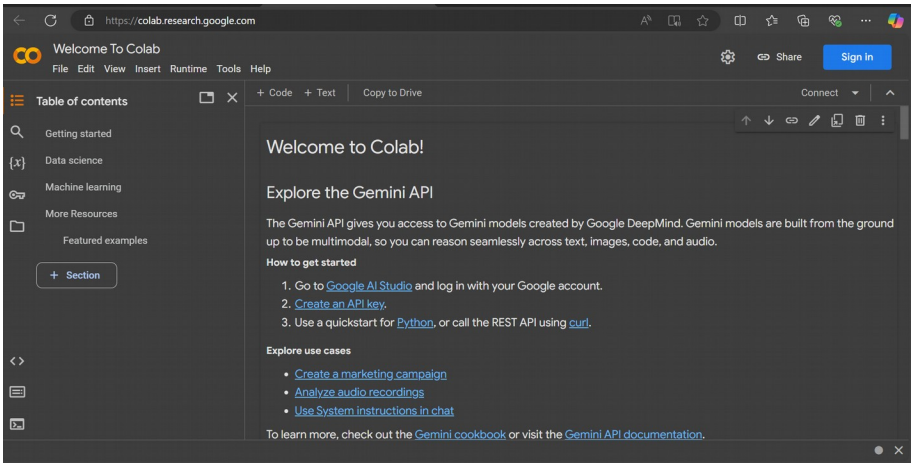
Sign up for Google Colab

Overview

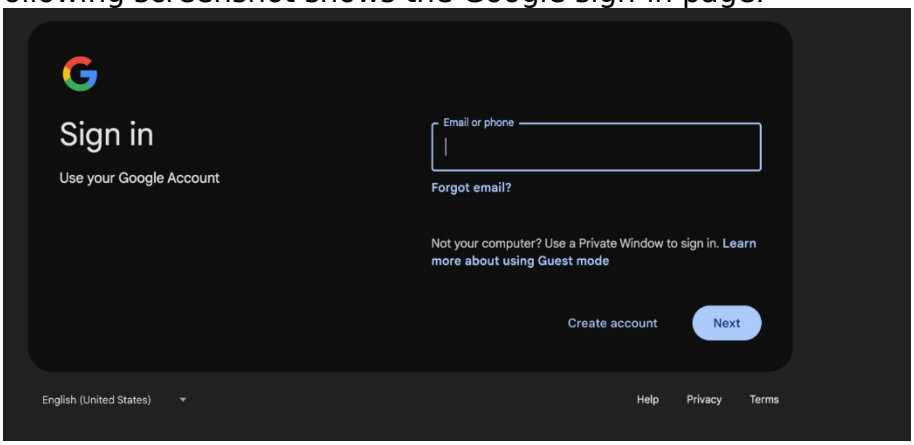
In this procedure, you'll set up a Google Colab account. Google Colab is a free cloud platform that lets you run code in notebooks, which are commonly used for machine learning, data science, and AI tasks. A Google Colab account allows you to install and use the tools needed to complete this lab.

Instructions

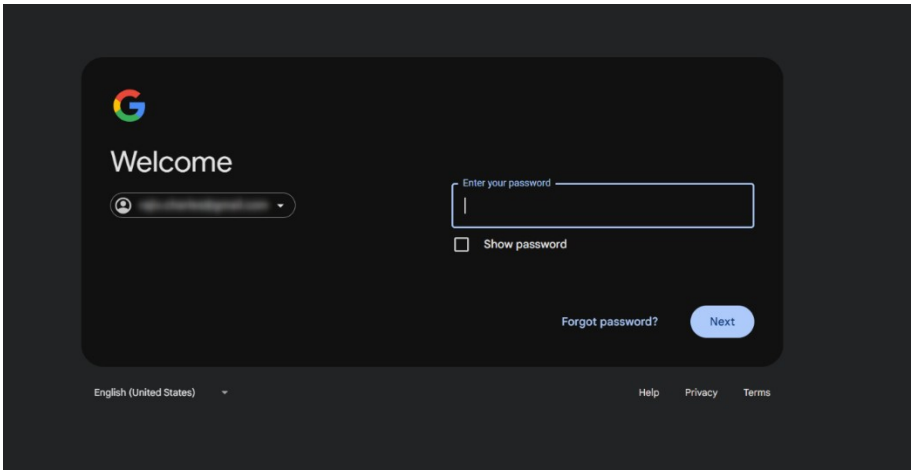
1. To sign up, go to the Google Colab website.
2. Select **Sign in**, as shown in the following screenshot.



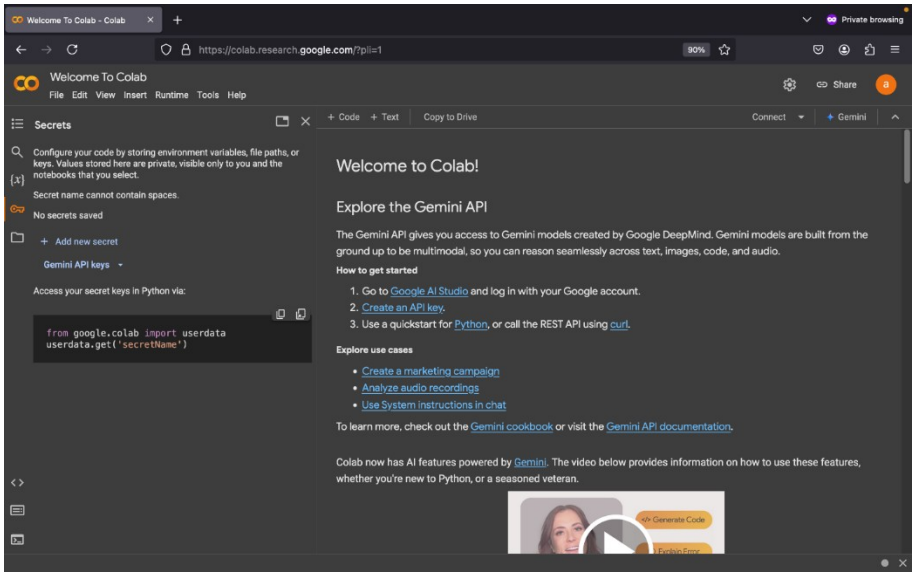
3. Enter your email or phone number in the **Email or phone** field, and then select **Next**.
   The following screenshot shows the Google sign-in page.



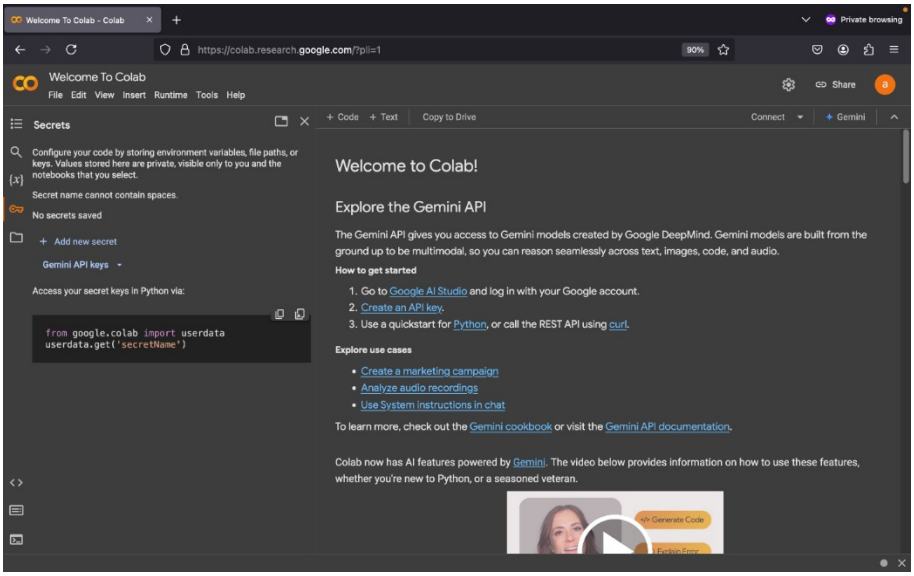
4. Enter your password in the **Enter your password** field, and then select **Next**.



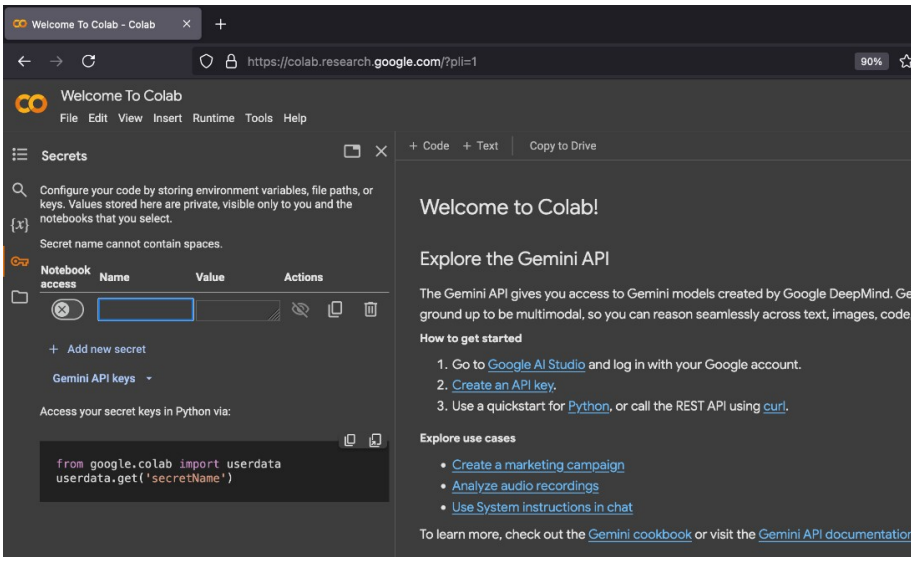
5. To store your Replicate API token, select the key icon from the **Secrets** tab on the Welcome to Colab page, as shown in the following screenshot.



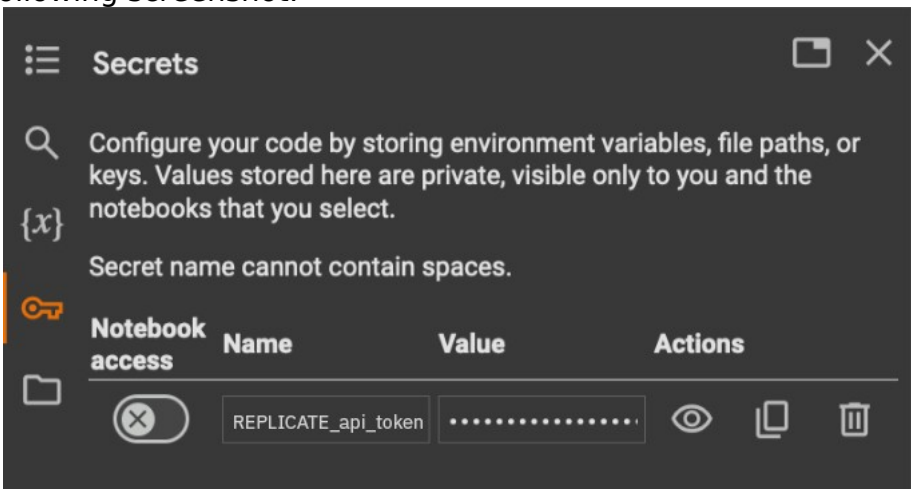
6. Select **Add new secret**.



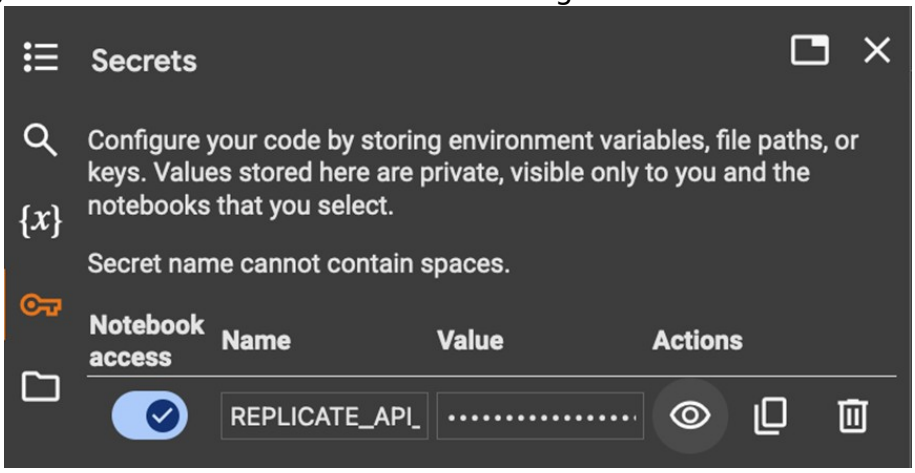
7. Type REPLICATE_api_token in the **Name** field.



8. Paste your Replicate API token you copied into the **Value** field, as shown in the following screenshot.



9. Set the **Notebook access** switch on, as shown in the following screenshot. Then, select the close icon to exit the configuration.



Prepare the Google Colab notebook with the necessary libraries

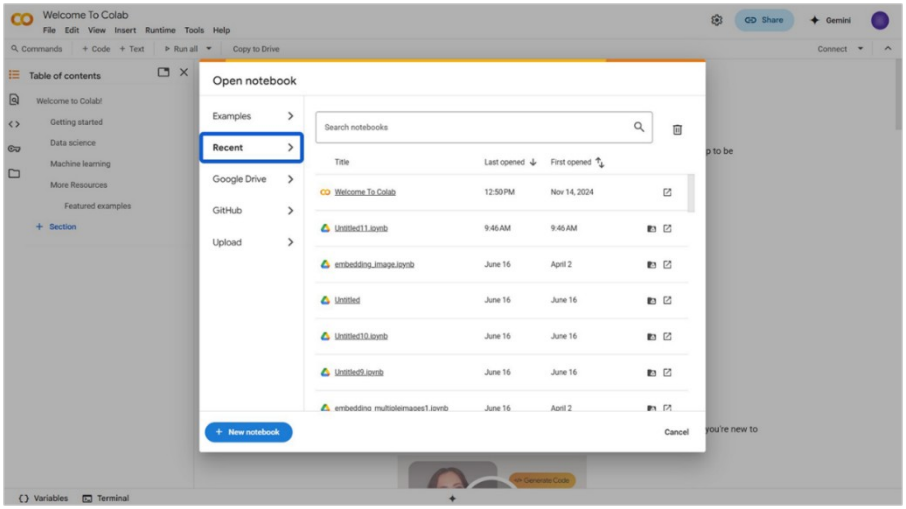
Overview

In this procedure, you'll create a new Google Colab notebook, install the required libraries, and import the necessary modules to set up your Colab environment for implementing the RAG system.

The document External KB for RAG.txt has some excerpts from GoTravel's policy documents. Save this document on your machine before you begin this lab.

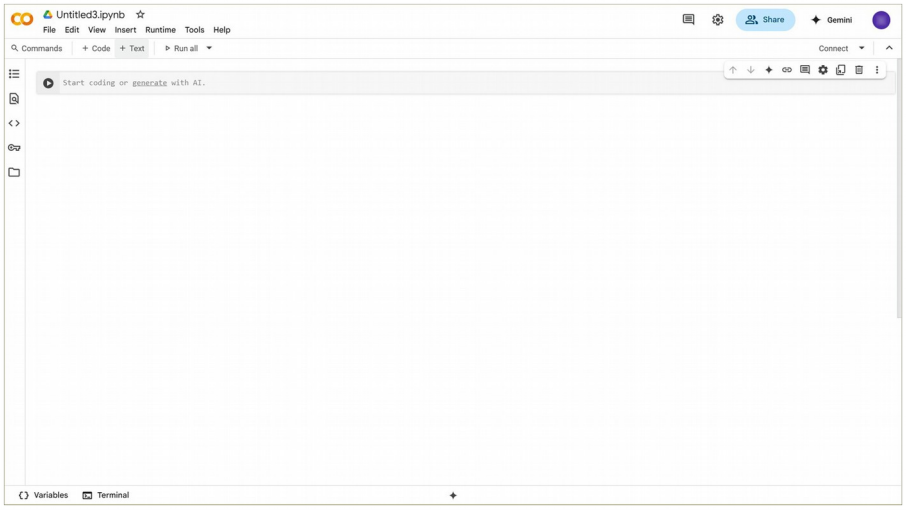
Instructions

1. Go to the Google Colab website. The Open notebook window is displayed.



2. Select **New notebook**.

   Result: A new notebook with the first code cell is displayed, as shown in the following screenshot.

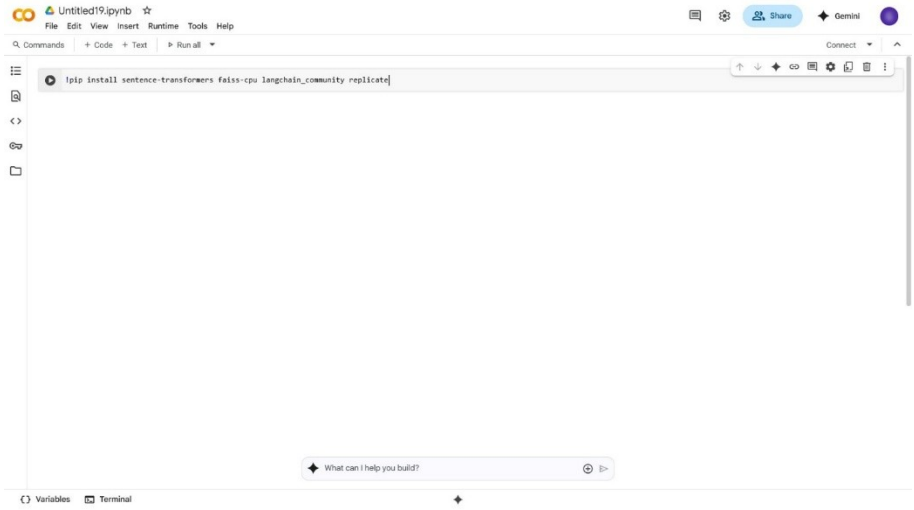


3. In order that GoTravel's RAG system can retrieve accurate information and generate consistent, context-aware responses, you need to install four Python libraries: sentence-transformers, faiss-cpu, langchain_community, and

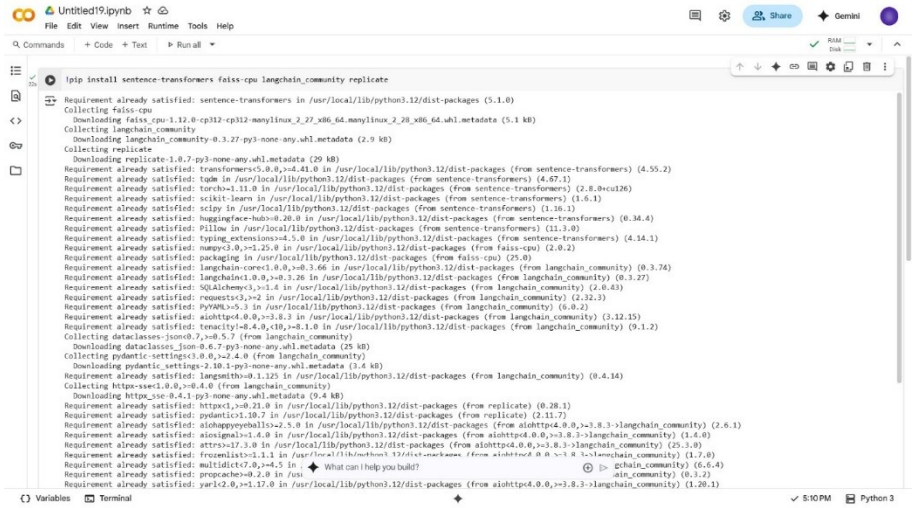
replicate. These libraries might not be available in Google Colab by default. To install them, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute it.

```
#pip install sentence-transformers faiss-cpu langchain_community replicate
```

The following screenshot has the code pasted into the code cell.



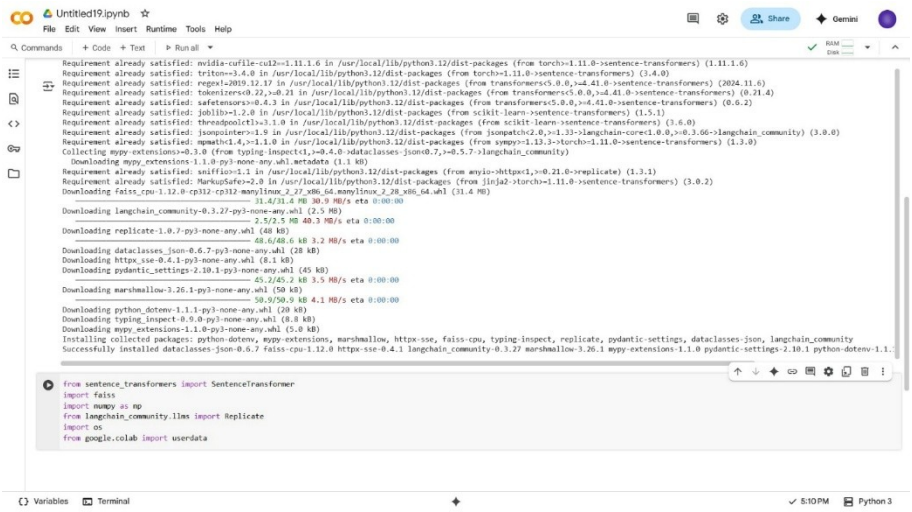
Result: The model generates the output that confirms you successfully installed required libraries as shown in the following screenshot.



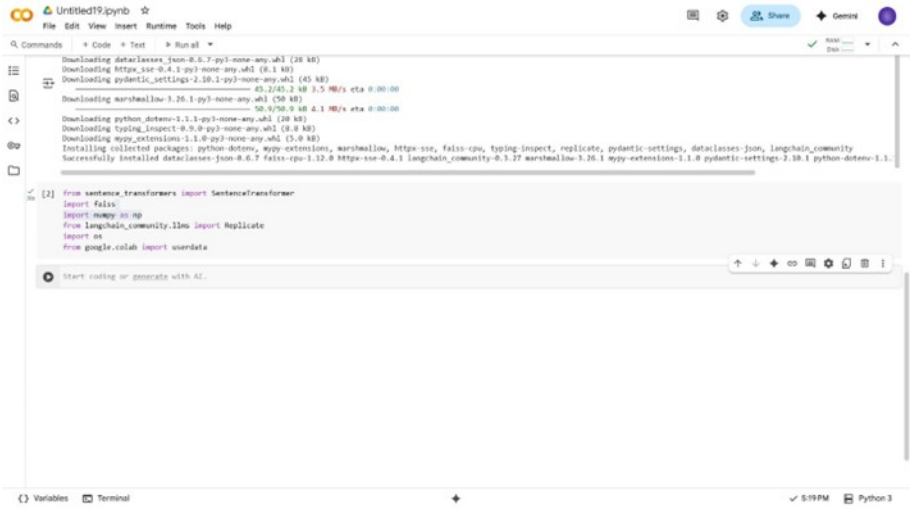
4. Import the installed libraries to load them into your notebook. To do this, copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
#from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from langchain_community.llms import Replicate
import os
from google.colab import userdata
```

The following screenshot shows the code pasted into the code cell.



Result: There isn't a visible output because this code only loads the libraries into the notebook's memory and sets up the environment for using the Replicate model. A green checkmark next to the Play icon indicates that the code ran and you successfully imported the necessary libraries.



Configure and test the LLM for baseline responses

Overview

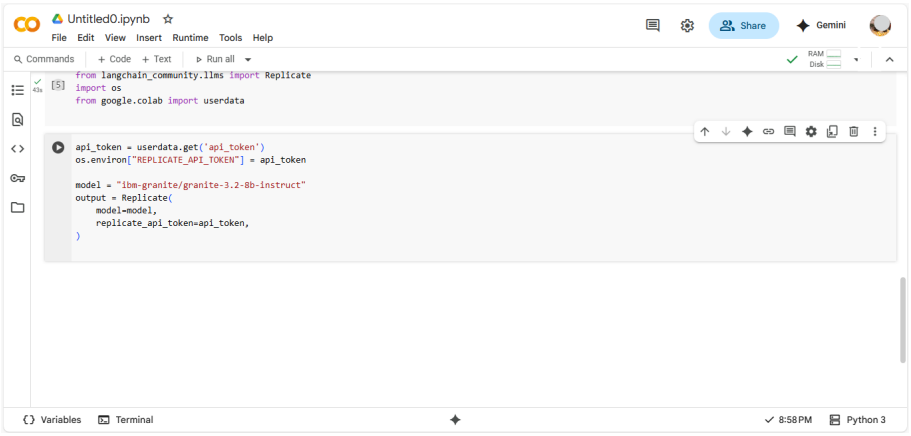
In this procedure, you'll connect to the Replicate application programming interface (API) and set up the IBM Granite language model, which will serve as the LLM powering GoTravel's travel assistant. You'll then run a baseline user query to review how the LLM responds using only its built-in knowledge.

Instructions

1. Begin configuring the IBM Granite model by authenticating it with the Replicate API and initializing it. To do this, copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
#api_token = userdata.get('api_token')
os.environ["REPLICATE_API_TOKEN"] = api_token
model = "ibm-granite/granite-3.2-8b-instruct"
output = Replicate(
    model=model,
    replicate_api_token=api_token,
)
```

The following screenshot shows the code pasted into the code cell.



Note: The code added in this step authenticates using the Replicate API and initializes the IBM Granite language model for text generation as follows:

- api_token = userdata.get('api_token') retrieves your Replicate API token securely from Google Colab's user data secrets. It’s important to store sensitive information such as API keys in secrets rather than directly in the code.
- os.environ["REPLICATE_API_TOKEN"] = api_token sets the retrieved API token as an environment variable named REPLICATE_API_TOKEN. Many libraries and tools that interact with Replicate look for this environment variable to authenticate.
- model = “ibm-granite/granite-3.2-8b-instruct” specifies the language model from Replicate that you want to use; in this case, it's the “granite-3.2-8b-instruct” model from “ibm-granite” family of LLMs.
- output = Replicate(model=model, replicate_api_token=api_token,) initializes an instance of the Replicate class from langchain_community. It passes the specified model name and the api_token for authentication. This output object can then be used to invoke the Replicate model for generating text.

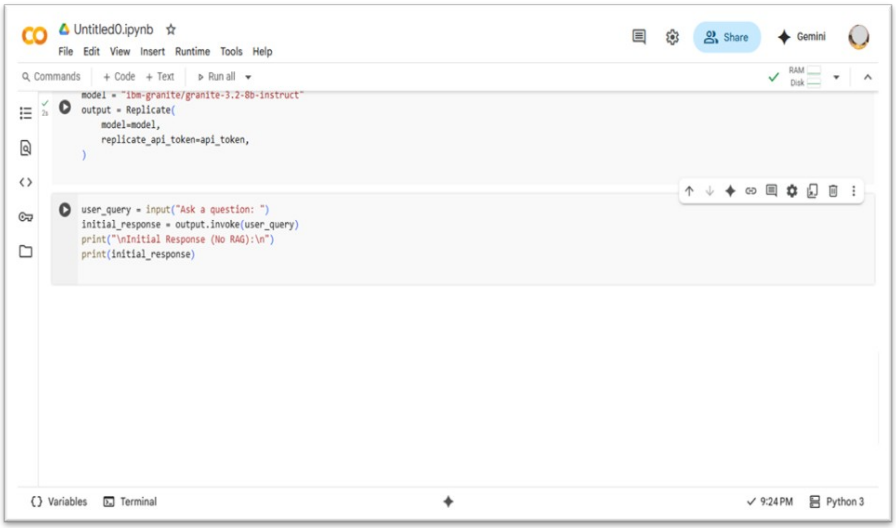
Result: There isn’t a visible output because this code only initializes the LLM. A green checkmark next to the Play icon indicates that the code ran successfully.

2. Assess the LLM’s baseline response to a user query. To do this, copy the following code, paste it into a code cell and then select the **Play** icon to execute it.

```
#user_query = input("Ask a question: ")
```

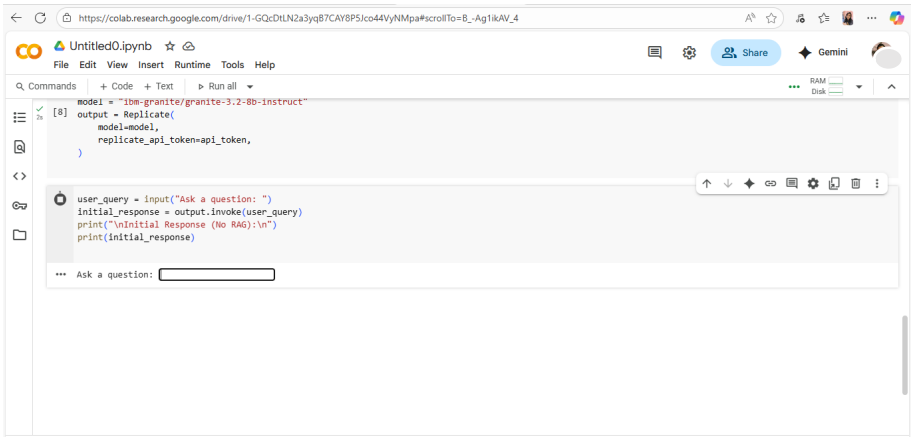
```
initial_response = output.invoke(user_query)
print("\nInitial Response (No RAG):\n")
print(initial_response)
```

The following screenshot shows the code pasted into the code cell.

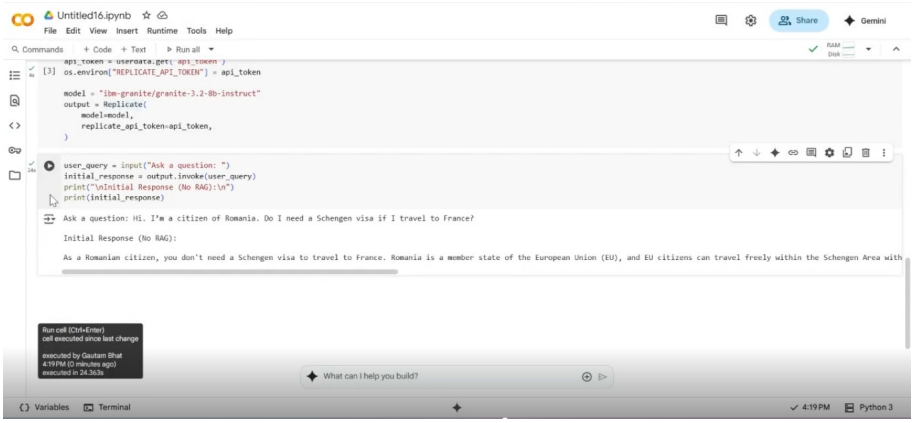


Result: On running the code, the “Ask a question” field is displayed.

3. In the **Ask a question** field type: Hi. I’m a citizen of Romania. Do I need a Schengen visa if I travel to France? and then, press **Enter**.



Result: This screenshot displays the output generated by the IBM Granite language model. Note that in this step, the LLM has generated a baseline response using only its built-in knowledge.



Set up a RAG system for context-based answering

Overview

In this procedure, you'll upload GoTravel's internal documents containing company policies and guidelines. The document is then split into chunks, which are embedded and indexed in Facebook AI similarity search (FAISS), and used to provide relevant context that enhances the LLM's responses.

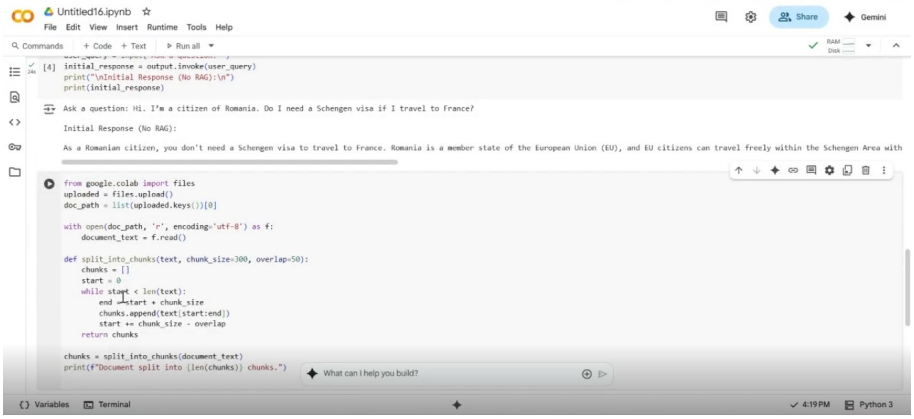
Note: For this procedure to work, you need to have the document External KB for RAG.txt saved on your machine. Please do so, if not already done.

Instructions

1. Upload GoTravel's internal documents to enhance LLM responses. For this, copy the following code, paste it into a new code cell, and then select the **Play** icon to execute it.

```
#from google.colab import files
uploaded = files.upload()
doc_path = list(uploaded.keys())[0]
with open(doc_path, 'r', encoding='utf-8') as f:
    document_text = f.read()
def split_into_chunks(text, chunk_size=300, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks
chunks = split_into_chunks(document_text)
print(f"Document split into {len(chunks)} chunks.")
```

The following screenshot shows the code pasted into the code cell.

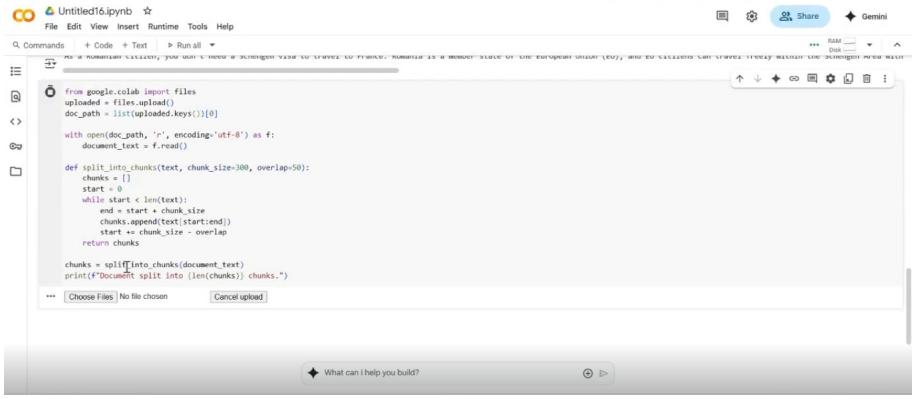


Note: In this step, you're writing the code that handles the file upload and prepares GoTravel's reference document for information retrieval so the RAG system can provide accurate and context-aware responses to customer queries. The code includes the following sections:

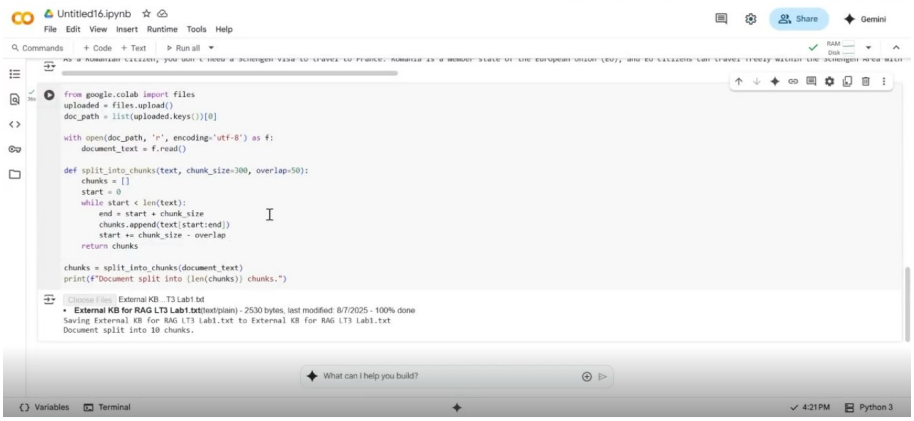
- **File upload:** Uses google.colab.files.upload() to select and upload a file from your local machine.
- **Read file content:** Opens the uploaded file in Unicode Transformation Format – 8 Bits (UTF-8), an encoding format that supports a wide range of characters and symbols.
- **Split into chunks:** Divides the document into smaller pieces of approximately 300 characters, with 50 characters overlapping between chunks to maintain context across boundaries.
- **Confirm readiness:** Prints the number of chunks created, confirming that the document is now ready for embedding and retrieval.

Result: After the code executes, the file upload option is displayed.

2. To upload the document External KB for RAG.txt from your machine, select **Choose Files**. Browse to the location where you have saved it and select **Open**.



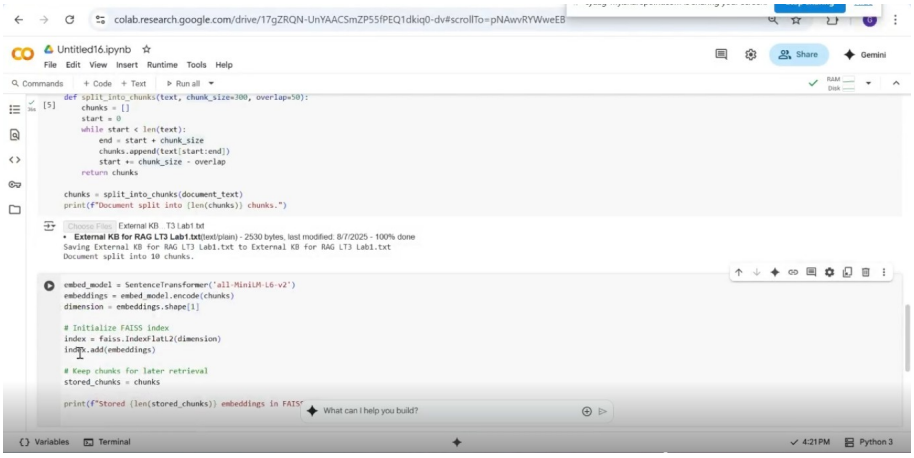
Result: After you upload the file External KB for RAG.txt it is divided into overlapping chunks. A message confirming the number of chunks created is displayed, as shown in the following screenshot.



3. Convert these chunks into vector embeddings and store them in a searchable FAISS index so that your Travel Assistant can retrieve relevant context and provide accurate, context-aware responses. To do this, copy the following code, paste it into a new code cell, and then select the **Play** icon to execute it.

```
#embed_model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = embed_model.encode(chunks)
dimension = embeddings.shape[1]
# Initialize FAISS index
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
# Keep chunks for later retrieval
stored_chunks = chunks
print(f"Stored {len(stored_chunks)} embeddings in FAISS index.")
```

The following screenshot shows the code pasted into the code cell.

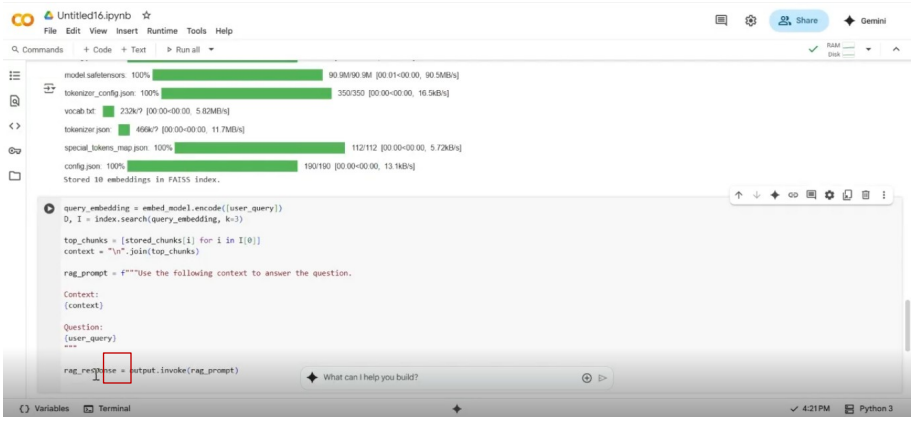


Result: There won't be a visible output. However, the green checkmark beside the Play icon confirms the creation of vector embedding and set up of FAISS index.

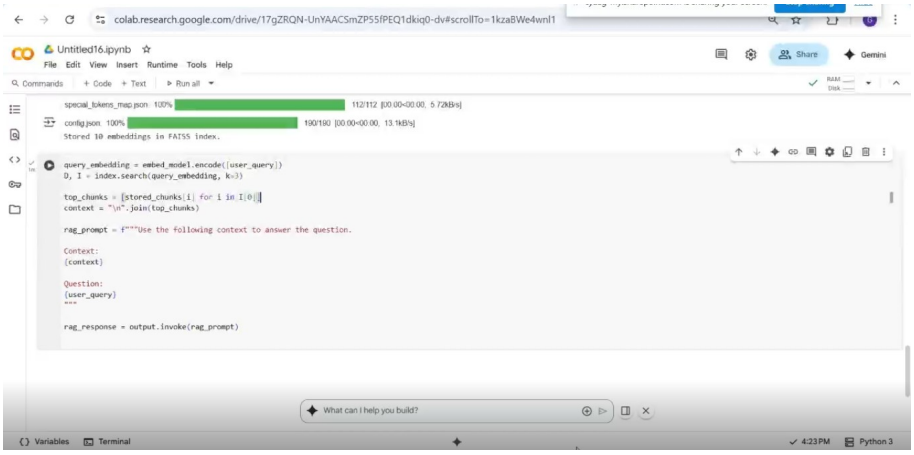
4. Retrieve relevant chunks from FAISS to generate RAG-enabled response. To do this, copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
# embed_model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = embed_model.encode(chunks)
dimension = embeddings.shape[1]
# Initialize FAISS index
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
# Keep chunks for later retrieval
stored_chunks = chunks
print(f"Stored {len(stored_chunks)} embeddings in FAISS index.")
```

The following screenshot shows the code pasted into the code cell.



Result: A green check mark appears next to the Play icon, confirming that the retrieval and RAG response generation code executed successfully.



Compare baseline responses with context-augmented answers

Overview

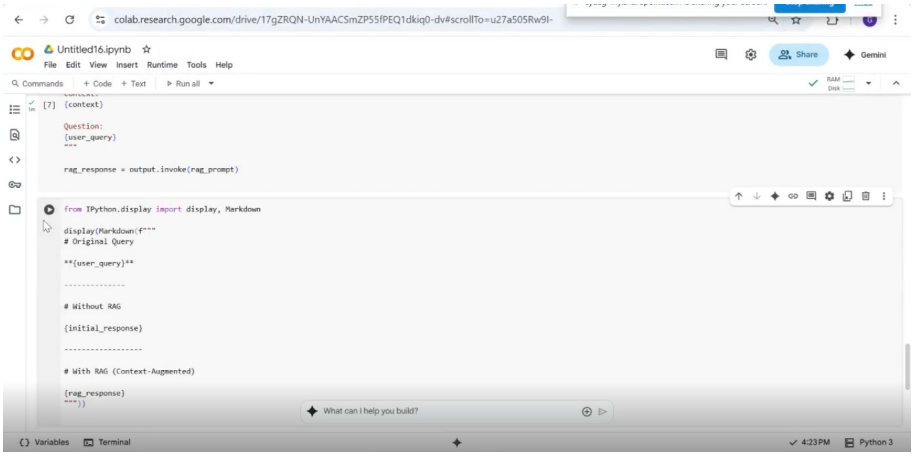
In this procedure, you'll compare the LLM's baseline response and RAG-enhanced response to assess whether real-time access to GoTravel's internal documents improved the accuracy and relevance of responses to customer queries.

Instructions

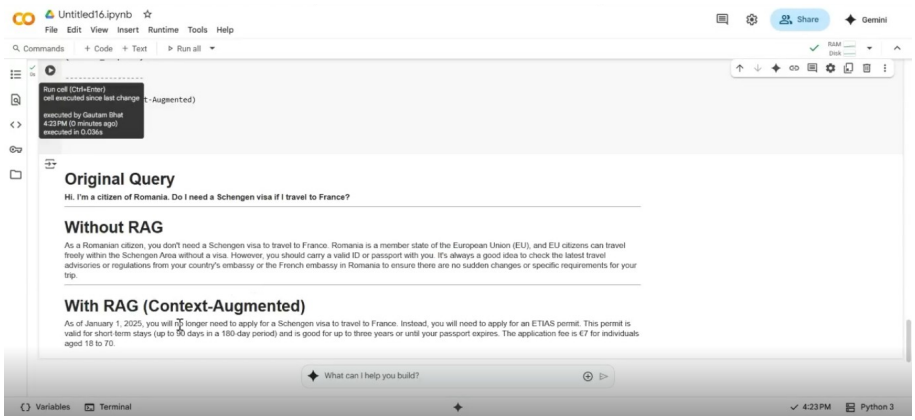
1. Compare the response to a baseline query with RAG-enabled responses. To do this, copy the following code, paste it into a new code cell, and then select the **Play** icon to execute it.

```
#from IPython.display import display, Markdown
display(Markdown(f"""
# Original Query
**{user_query}**
-------------
# Without RAG
{initial_response}
-----------------
# With RAG (Context-Augmented)
{rag_response}
"""))
```

The following screenshot shows the code pasted into the code cell.



Result: The following screenshot shows the user query, the initial baseline response, and the RAG-enhanced response. Observe the difference in responses. Notice that with RAG, the response is more reliable and contextually relevant.



Conclusion

Congratulations! In this lab, you started by preparing the Google Colab notebook with the necessary libraries. Then, you configured and tested the LLM for baseline responses. Next, you uploaded GoTravel's internal documents to set up the RAG system. Finally, you compared baseline responses with context-augmented responses. By combining an LLM with GoTravel's policy documents, you've enabled the Travel Assistant to deliver faster, more accurate, and context-aware answers to customer queries.